Fyzická organizácia dát a optimalizácia dotazov

Ján Mazák

FMFI UK Bratislava

Logická organizácia dát

Dáta v relačnom modeli:

- hodnoty jednotlivých atribútov sú zoskupené do záznamov
- záznamy tvoria reláciu
- záznamy môžu mať fixnú alebo variabilnú dĺžku

Ako toto fyzicky reprezentovať na disku?

Fyzická organizácia dát

Neusporiadané záznamy v spájanom zozname:

- v každom diskovom bloku niekoľko záznamov a pointer na ďalší blok
- sekvenčné vyhľadávanie; trvá $O(n)$, čiže dlho
- vkladanie sa dá urýchliť, ak osobitne evidujeme plné bloky a bloky s prázdnym miestom, v ideálnom prípade je to $O(1)$
- mazanie záznamov môže spôsobiť fragmentáciu a predĺžiť vyhľadávanie neúmerne množstvu uložených dát

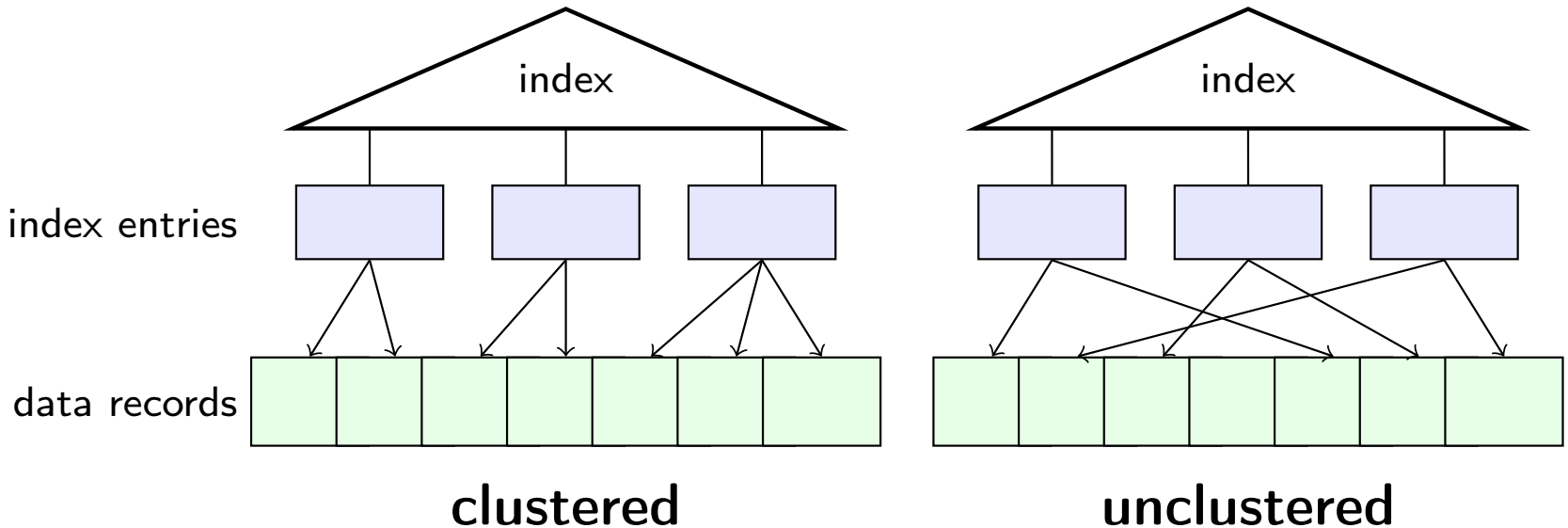
Fyzická organizácia dát

Usporiadané záznamy v spájanom zozname:

- záznamy usporiadané podľa nejakého kľúča, v každom diskovom bloku niekoľko záznamov a pointer na ďalší blok
- binárne vyhľadávanie sotva použiteľné, keďže bloky s dátami nie sú na disku uložené súvisle
- vkladanie v najhoršom prípade $O(n)$, lebo treba spraviť miesto na nový záznam odsunutím existujúcich
- ak evidujeme osobitne usporiadané kľúče s odkazmi na blok, ktorý záznamy s danou hodnotou kľúča obsahuje, zrýchlime vyhľadávanie — indexovanie záznamov

Index

Výhody clustered indexu: rýchlejší prístup k okolitým záznamom, napr. ak hľadáme dáta z daného intervalu. Údržba indexu (odstraňovanie „unclusterizácie“) však niečo stojí.

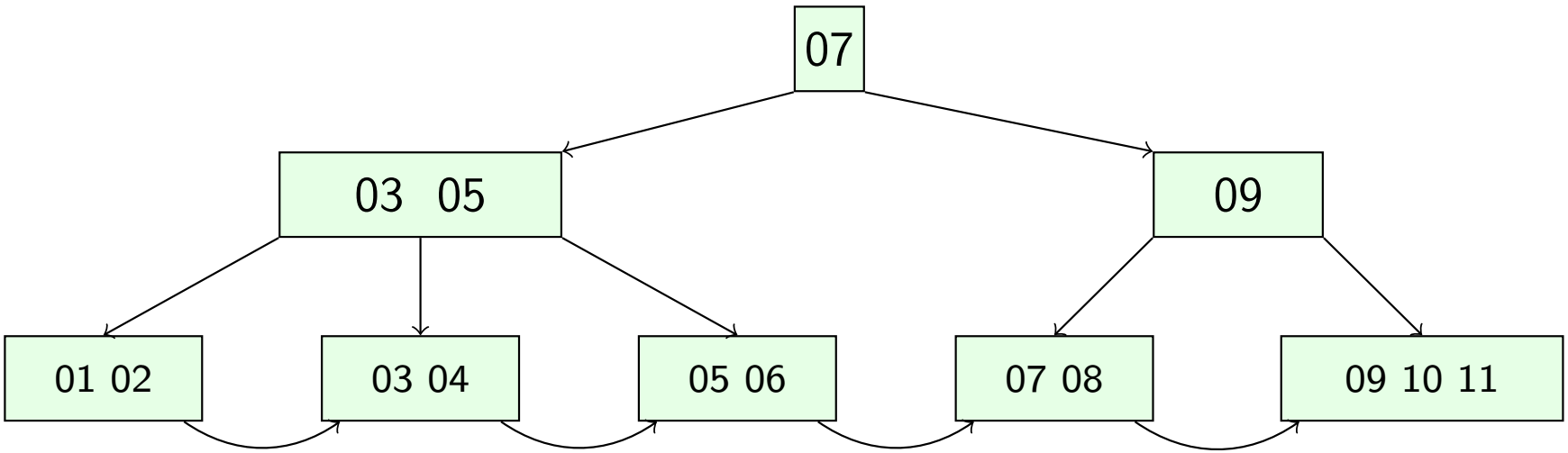


B+ strom

B+ strom: vyvážený strom vhodný pre uloženie na disku.

- záznamy (resp. odkazy na ne) sú uložené v listoch
- parametrizovaný hodnotou m tak, aby stupeň vetvenia bol medzi $m/2$ a m
- voľba m podľa veľkosti bloku (rádovo stovky)
- malá výška stromu, zhruba $\log_m(n)$
- minimalizujeme počet blokov, čo treba načítať pri prístupe k listu
- vyhľadávanie, vkladanie, mazanie $O(\log_m(n))$

B+ strom



Vyskúšajte si:

`https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html`

Index

B+ strom by mohol obsahovať hodnoty priamo v nelistových uzloch (B strom). Neefektívne: pri traverzovaní stromu zbytočne čítame dáta, ktoré nás nezaujímajú.

Ak sú v listoch len referencie, ako uložiť záznamy do blokov?

- PostgreSQL: záznamy sú uložené bez udržiavaného usporiadania podľa kľúča; nový riadok môže použiť voľné miesto v existujúcom bloku (správu voľného miesta rieši databáza). Príkaz `CLUSTER tabulka USING index` tabuľku raz prepíše podľa indexu; pri ďalších zmenách sa toto poradie postupne rozpadá.
- SQLite: tabuľka je B+ strom zoradený podľa rowid (pri `INTEGER PRIMARY KEY` je to ten kľúč). Listy obsahujú celé riadky, takže samotná tabuľka je clustered index. Ďalší index obsahuje hodnotu a rowid, riadok nájde až cez strom tabuľky.

VACUUM — PostgreSQL

- `UPDATE` a `DELETE` nechávajú staré verzie riadkov (kvôli MVCC); `VACUUM` ich odstráni a miesto označí na opätovné použitie.
- Bežný `VACUUM` súbor tabuľky zvyčajne nezmenší, nebráni čítaniu ani zápisu.
- `VACUUM FULL` tabuľku prepíše a miesto vráti operačnému systému; počas behu však blokuje aj čítanie tabuľky.
- Riadky neusporadúva (na to slúži `CLUSTER`).
- Spúšťa sa aj automaticky (autovacuum).
- Z naivného sledovania diskového priestoru nemožno robiť závery o činnosti DBMS.

  Pokus: 20 000 riadkov (4,7 MB), zmazanie polovice a 1 000 nových riadkov: 4,9 MB; `VACUUM`: 4,9 MB; `VACUUM FULL`: 2,6 MB.

VACUUM — SQLite

- `DELETE` uvoľní prázdne bloky do zoznamu voľných; ďalšie vkladanie ich použije. Súbor sa nezmenší.
- `VACUUM` prekopíruje databázu do nového súboru, odstráni voľné bloky a zmenší fragmentáciu.
- `VACUUM` vyžaduje dočasne ďalšie miesto (až dvojnásobok) a počas svojej činnosti zamkne databázu.
- Pozor: v tabuľkách bez `INTEGER PRIMARY KEY` sa môžu zmeniť rowid.

Hashovanie

Aj hashovanie možno použiť na vytvorenie indexu.

Pri dobrej hashovacej funkcii možno očakávať vyhľadávanie v konštantnom čase, ak index nie je *preplnený* — ak je záznamov priveľa, v každom hashovacom buckete je pridlhý spájaný zoznam. Vtedy možno buckety rozdeliť použitím dodatočnej hashovacej funkcie. Existujú tiež spôsoby hashovania, ktoré priebežne pridávajú buckety.

Nevýhoda: len podmienka s rovnosťou, nie nerovnosť či intervaly.

Hashovanie

Hashovanie možno použiť na horizontálne škálovanie.

Jednotlivé hash buckety môžeme skladovať na osobitných serveroch. Vyhľadávanie podľa kľúča aj join s podmienkou na rovnosť tak možno naďalej počítať efektívne.

Pri odobraní alebo pridaní servera však máme problém. Riešenie: *consistent hashing*. (Po kliknutí na link sa dozviete viac, neočakáva sa však, že to budete vedieť v rámci tohto predmetu. Využité napr. v DynamoDB a Cassandra.)

Ďalší problém je *workload skew* — ak majú hodnoty hashovaného atribútu nerovnomernú distribúciu (napr. polovica záznamov rovnakú), nepomôže ani dobrá hashovacia funkcia. Riešenie: link.

Vytvorenie indexu

Hash index na vyhľadávanie podľa emp_id:

```
CREATE INDEX ON employee USING HASH (emp_id);
```

Index na zabezpečenie unikátnosti mena (B+ strom):

```
CREATE UNIQUE INDEX i1 ON employee (name);
```

Index pre vypočítanú hodnotu (nie priamo atribút relácie):

```
CREATE INDEX ON employee (lower(name));
```

Index obsahujúci dodatočné dáta (podľa salary nemožno vyhľadávať, iba podľa emp_id a name):

```
CREATE INDEX ON employee (emp_id, name) INCLUDE (salary);
```

Fyzické operátory — table scan

- sequential (linear) scan — prejdeme priamo záznamy zaradom, ako sú uložené (efektívne pre malé relácie)
- index scan — využije sa existujúci index, neraz netreba vidieť všetky záznamy, lebo máme podmienku za WHERE
- index-only scan — k záznamom netreba vôbec pristupovať, ak nás zaujímajú len atribúty obsiahnuté v indexe
- bitmap index scan vytvorí bitmapu umiestnení vyhovujúcich riadkov; bitmap heap scan potom číta príslušné dátové bloky v poradí ich uloženia

Bitmap scan: prečo a kedy?

- Index môže vracať odkazy na riadky v blokoch $7, 2, 7, 2, 9$. Bitmap heap scan navštívi bloky $2, 7, 9$; odkazy zoskupí podľa blokov.
- Plánovač odhaduje počet vyhovujúcich riadkov zo štatistík z `ANALYZE`: pozná aj najčastejšie hodnoty a približné rozdelenie hodnôt.
- Pri veľmi malom výbere často vyhrá index scan, pri väčšom bitmap scan, pri čítaní veľkej časti tabuľky sequential scan. Záleží aj na rozmiestnení riadkov a cene prístupov.

Pri nedostatku pamäte môže bitmapa označovať iba celé bloky; po ich načítaní treba podmienku znovu overiť na riadkoch.

Druhy indexov

Druhy indexov:

- btree (default)
- hash
- gist (geografické dáta o polohe)
- GIN (Generalized Inverted Index): pre hodnoty zložené z prvkov (polia, `jsonb`, fulltext)
- bloom filter („user-installed“ in PostgreSQL)
- k-dimensional tree, ...

Indexy zrýchľujú vyhľadávanie, ale spomaľujú vkladanie.
Pri vkladaní veľkého množstva dát sa oplatí index zrušiť a potom nanovo vybudovať.

GIN — invertovaný index

- Pre každú hodnotu uloží zoznam riadkov, ktoré ho obsahujú v danom kompozitnom atribúte.
- Dotaz @> nájde kandidátov prienikom zoznamov a potom overí podmienku (Recheck Cond).
- Cena: pomalší zápis, lebo sa udržiavajú zoznamy.

```
CREATE INDEX request_log_gin ON request_log
  USING GIN (payload);
EXPLAIN SELECT id FROM request_log
WHERE payload @> '{"client": {"ip": "10.0.0.7"}}';

Bitmap Heap Scan on request_log  (cost=17.32..21.33)
  Recheck Cond: (payload @> '...'::jsonb)
  ->  Bitmap Index Scan on request_log_gin
        Index Cond: (payload @> '...'::jsonb)
```

GIN — Recheck Cond

- GIN ukladá kľúče a hodnoty osobitne, nie cesty k nim.
- Riadok s `ip` = `10.0.0.7` na vrchu a `ip` = `10.0.0.9` pod `client` obsahuje všetky prvky dotazu: kľúče `client`, `ip` a hodnotu `10.0.0.7`.
- Index ho vráti ako kandidáta, ale nie je správny: `10.0.0.7` nie je pod `client`.
- Recheck Cond overí `@>` na skutočnom riadku a kandidáta vyradí.

```
Bitmap Heap Scan on request_log (actual loops=1)
  Recheck Cond: (payload @> '{"client": {"ip": "10.0.0.7"}}'::jsonb)
  Rows Removed by Index Recheck: 1
  ->  Bitmap Index Scan on request_log_gin (actual loops=1)
```

Životný cyklus dotazu

1. Query parsing and translation
   - syntax check — Je to korektné SQL?
   - user privilege check — Má používateľ oprávnenie pristupovať k db objektom?
   - semantic check — Existujú relácie a atribúty? Sedia dátové typy (napr. pri EXCEPT)?
   - cache check — Ak dotaz neprišiel prvýkrát, možno máme v cache už optimalizovaný preklad do relačnej algebry/fyzických operátorov. Ak nie, preložiť.
   - preklad do relačnej algebry

Životný cyklus dotazu

2. Query optimization
   - Zozbieranie štatistických dát o reláciách (napr. odhad počtu záznamov).
   - Preusporiadanie operátorov relačnej algebry, napr. identifikácia viacnásobného výpočtu toho istého poddotazu či použitie rôznych heuristík.
   - Preklad operátorov relačnej algebry do fyzických operátorov.
   - Možnosť zvážiť niekoľko rôznych zápisov výpočtu, odhad času, výber najlepšej možnosti (nie nutne najrýchlejšej, možno tiež optimalizovať pamäť).
   - Vytvorenie plánu pre výpočet dotazu (query plan, používateľ má možnosť zobraziť pomocou EXPLAIN).

Životný cyklus dotazu

3. Query execution
   - FROM, JOIN
   - WHERE
   - GROUP BY
   - HAVING
   - SELECT (projection, window functions, CASE, ...)
   - DISTINCT
   - ORDER BY
   - LIMIT / OFFSET

Čas výpočtu dotazu možno získať cez EXPLAIN ANALYZE.

Životný cyklus dotazu

4. Transmitting results
   - výsledok výpočtu možno uložiť na disk alebo priebežne posielať používateľovi
   - rozsiahlejšie výsledky nemožno poslať naraz
   - typicky zahŕňa manažment sieťového pripojenia
   - z pohľadu programovacieho jazyka práca s objektmi ako *cursor*

Heuristiky pre join

Pre optimalizáciu výpočtu joinov existuje rozsiahla literatúra — ide o kľúčovú operáciu. Pár ukážok heuristík (plán výpočtu musíme zvoliť „okamžite“, nemožno vyriešiť NP-ťažké problémy):

- odsunúť projekcie a selekcie čo najnižšie
  (zmenšuje medzivýsledky; vďaka projekcii sa do diskového bloku vojde viac záznamov)
- uvažovať len o left-deep plans:

$$(((A_1 \bowtie A_2) \bowtie A_3) \bowtie A_4) \ldots$$

  (znižuje počet možných plánov na $n!$; pri takýchto plánoch možno ľahko uplatniť pipelining — nemusíme ukladať medzivýsledky na disk)

Heuristiky pre join

- počítať karteziánsky súčin, len keď sa tomu nedá vyhnúť (join bez podmienok býva veľký)
- „uši v hypergrafe“ rátať až na konci (keď nejaká relácia má atribúty z joinovacích podmienok spoločné len s jednou inou, odsunúť ju vo výpočte joinu na neskôr)

Ak potrebujete rátať join „manuálne“ (pre rozsiahle dáta mimo relačného DBMS), azda sa oplatí naštudovať si niečo o optimalizácii joinov.

Query plans — EXPLAIN

- Plánovanie výpočtu dotazu budeme sledovať pre databázu PostgreSQL.
- Príkaz EXPLAIN SELECT ... miesto výpočtu dotazu vráti plán jeho výpočtu zapísaný pomocou fyzických operátorov, aj s odhadmi času potrebného na jednotlivé kroky.
- Pri odhade ceny sa využívajú konfigurovateľné konštanty (napr. vyhodnotenie bežnej podmienky za WHERE je 400x rýchlejšie ako načítanie bloku z disku).
- Reálny čas výpočtu vieme získať pomocou EXPLAIN ANALYZE SELECT ... .
- SQLite má plán dotazu v príkaze `EXPLAIN QUERY PLAN`; samotný `EXPLAIN` vypíše bajtkód virtuálneho stroja, ktorý je oveľa menej zrozumiteľný.

Query plans — EXPLAIN

```
EXPLAIN
SELECT e.name
FROM employee e
WHERE e.salary > 1000
ORDER BY e.name;
```

```
------------------------------------------------------------------
 Sort  (cost=16.30..16.52 rows=90 width=118)
   Sort Key: name
   ->  Seq Scan on employee e  (cost=0.00..13.38 rows=90 width=118)
         Filter: (salary > '1000'::numeric)
```

Query plans — EXPLAIN

EXPLAIN uvádza odhadovanú veľkosť relácií. Je značne nepresná (odhad je 90 záznamov, v skutočnosti ich je < 20). Presná evidencia by viedla k spomaleniu INSERT/UPDATE/DELETE.

Ľavé číslo pri odhade ceny (cost) znamená čas do vypočítania prvého riadka výstupu (napr. pre operátor Sort sa celá práca musí vykonať pred vypísaním prvého riadka). Pravé číslo je čas ukončenia činnosti operátora. Udávané čísla sú kumulatívne (zahŕňajú aj čas výpočtu podriadených operátorov). Jednotky sú abstraktné (1 zodpovedá sekvenčnému načítaniu jednej stránky). `rows` je odhad počtu riadkov výstupu, `width` odhad veľkosti jedného riadku v bajtoch. V ďalších plánoch na slajdoch ich vynechávame, ale v reálnom výstupe EXPLAIN ich uvidíte.

Query plans — EXPLAIN

```
EXPLAIN
SELECT d.name, e.name
FROM department d
JOIN employee e ON d.name < e.name;

------------------------------------------------------------------
Nested Loop  (cost=0.00..1241.38)
Join Filter: ((d.name)::text < (e.name)::text)
  ->  Seq Scan on department d  (cost=0.00..13.00)
  ->  Materialize  (cost=0.00..14.05)
        ->  Seq Scan on employee e
            (cost=0.00..12.70)
```

Query plans — EXPLAIN

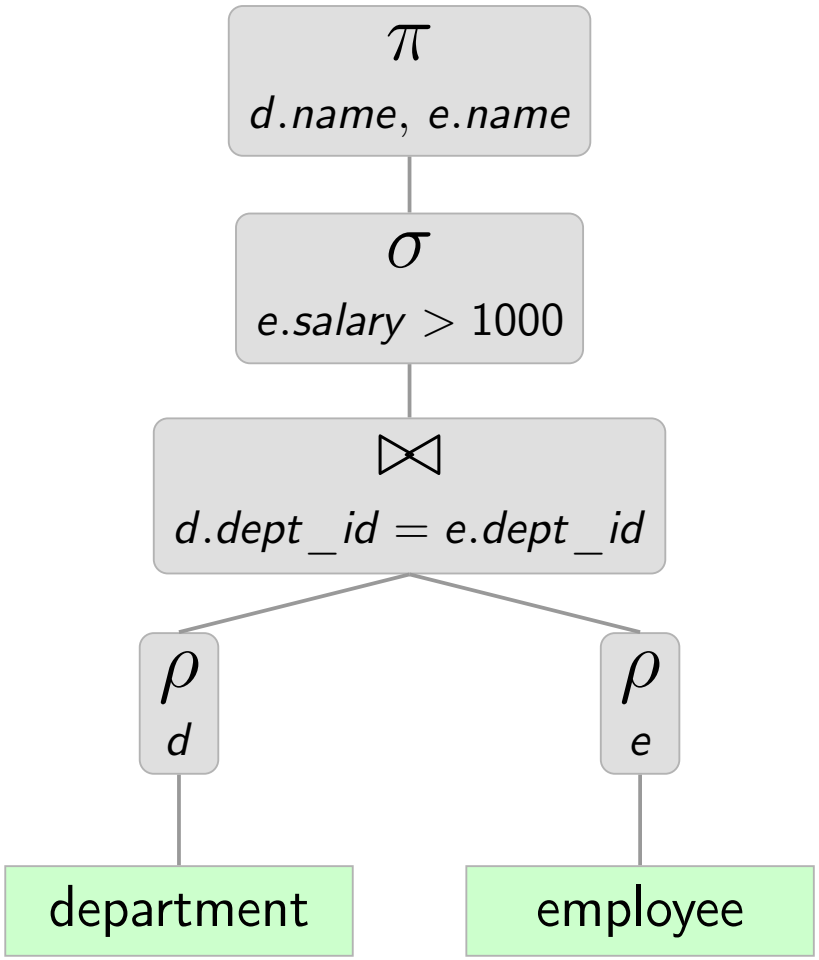
Počítame v PostgreSQL dotaz

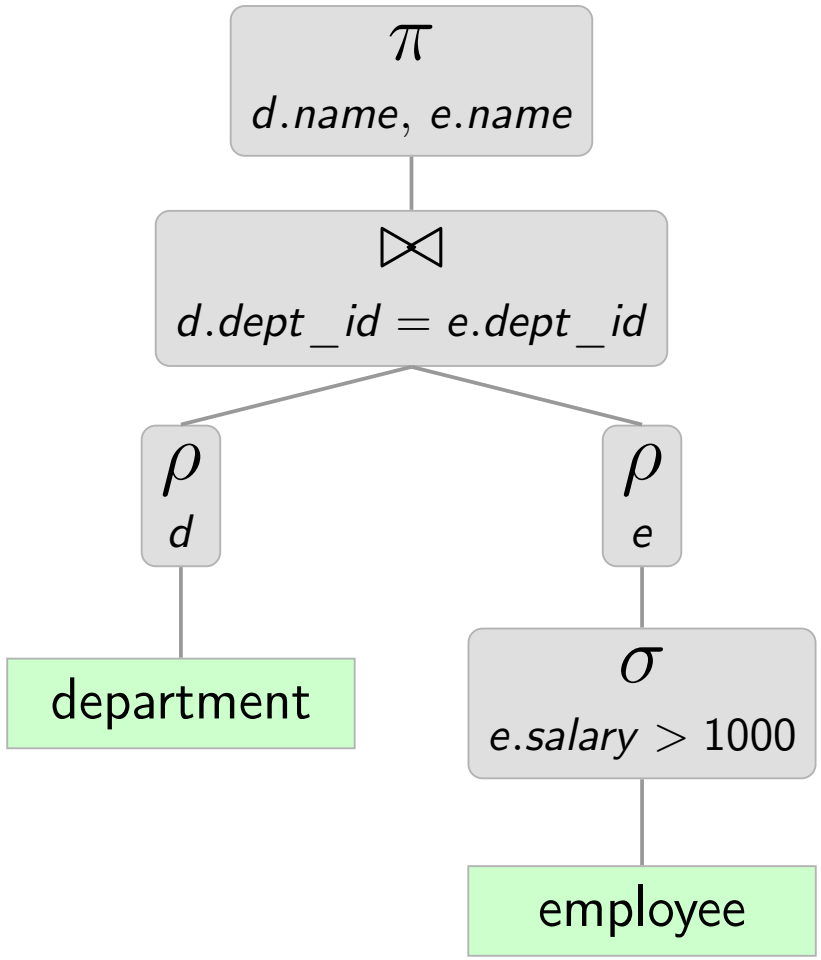
```
EXPLAIN
SELECT d.name, e.name
FROM department d
JOIN employee e ON d.dept_id = e.dept_id
WHERE e.salary > 1000;
```

Pre relácie employee a department sme nevytvorili žiaden index. Pozor: DBMS často vytvorí index automaticky (napr. pre `PRIMARY KEY`), v našej schéme však PRIMARY KEY nemáme. V tabuľkách je cca 10 záznamov.

Query plans — EXPLAIN

Oddelenia a mená zamestnancov s platom nad 1000.





Query plans — EXPLAIN

```
------------------------------------------------------------
Hash Join  (cost=16.75..30.36)
Hash Cond: (e.dept_id = d.dept_id)
  ->  Seq Scan on employee e  (cost=0.00..13.38)
        Filter: (salary > '1000'::numeric)
  ->  Hash  (cost=13.00..13.00)
        ->  Seq Scan on department d
            (cost=0.00..13.00)
```

Celá tabuľka je v jednom bloku, preto sekvenčný scan je rýchly.

Query plans — EXPLAIN

Do relácie department sme pridali milión riadkov a nastavili `PRIMARY KEY (dept_id)`.

```
------------------------------------------------------------------
Nested Loop  (cost=0.42..26.43)
  ->  Seq Scan on employee e  (cost=0.00..1.10)
        Filter: (salary > '1000'::numeric)
  ->  Index Scan using department_pkey on department d
      (cost=0.42..8.44)
        Index Cond: (dept_id = e.dept_id)
```

Zamestnancov vyhovujúcich filtru je málo. Pre každého vyhľadáme oddelenie cez index `department_pkey`, takže nemusíme prechádzať celú miliónovú tabuľku oddelení. Stále ide o nested loop join: index zrýchľuje jeho vnútorný cyklus.

Query plans — EXPLAIN

Do nového indexu dáme všetky údaje potrebné pre tento dotaz:

```
CREATE INDEX i3 ON department(dept_id) INCLUDE (name);
```

```
------------------------------------------------------------------
Nested Loop  (cost=0.42..14.43)
  ->  Seq Scan on employee e  (cost=0.00..1.10)
        Filter: (salary > '1000'::numeric)
  ->  Index Only Scan using i3 on department d
      (cost=0.42..4.44)
        Index Cond: (dept_id = e.dept_id)
```

Query plans — EXPLAIN

Do relácie employee sme pridali tisíc riadkov (department zostáva milión riadkov).

```
------------------------------------------------------------------------
Merge Join  (cost=21.92..22.73)
Merge Cond: (d.dept_id = e.dept_id)
  ->  Index Scan using department_pkey on department d
      (cost=0.42..31223.00)
  ->  Sort  (cost=21.46..21.48)
        Sort Key: e.dept_id
        ->  Seq Scan on employee e
            (cost=0.00..21.36)
              Filter: (salary > '1000'::numeric)
```

Query plans — EXPLAIN

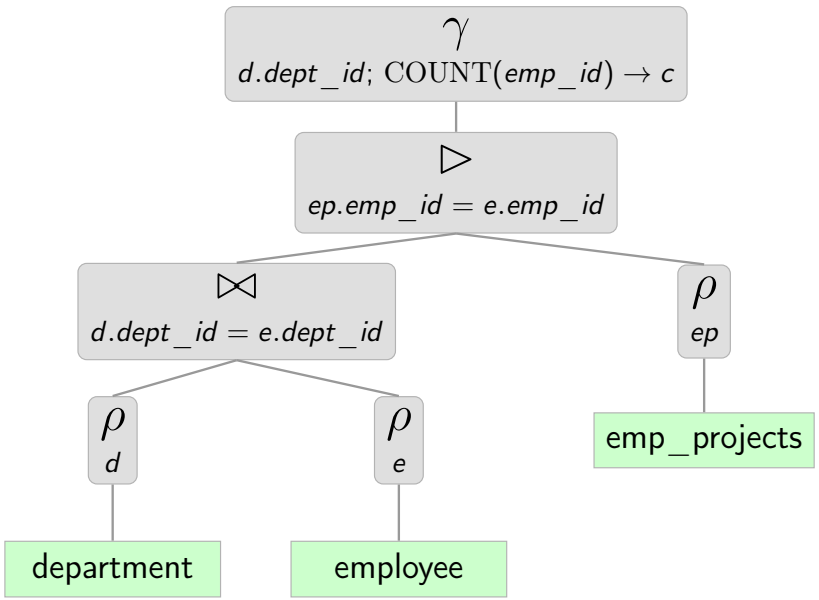
Vrátime sa k pôvodným malým tabuľkám. Počítame dotaz

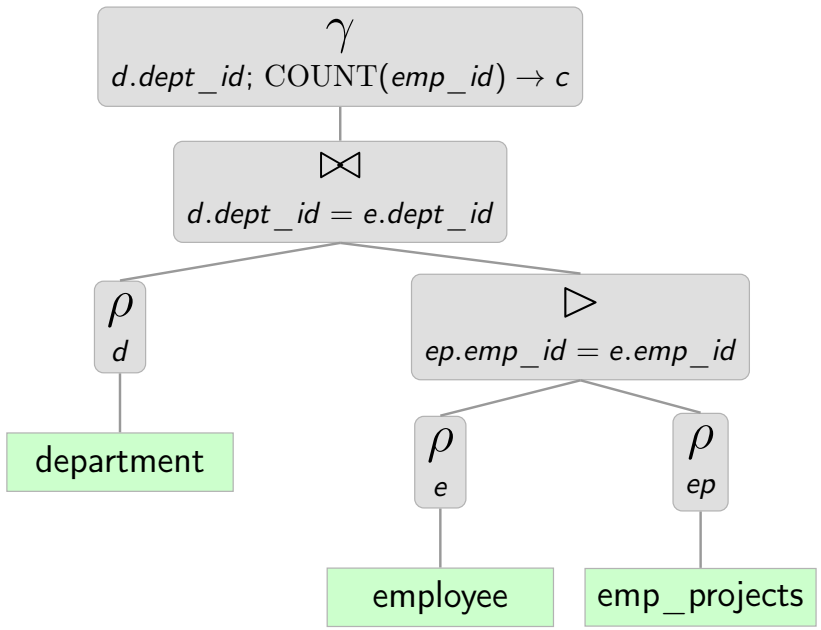
```
EXPLAIN
SELECT d.dept_id, COUNT(e.emp_id) AS c
FROM department d
JOIN employee e ON e.dept_id = d.dept_id
WHERE NOT EXISTS (
    SELECT 1
    FROM emp_projects ep
    WHERE ep.emp_id = e.emp_id
)
GROUP BY d.dept_id;
```

Pre uvedené relácie sme nevytvorili žiadne indexy.

Query plans — EXPLAIN

Počet zamestnancov bez projektu na jednotlivých oddeleniach.





Query plans — EXPLAIN

Fyzický plán (query plan):

```
HashAggregate  (cost=3.55..3.59)
  Group Key: d.dept_id
  ->  Hash Join  (cost=2.27..3.52)
        Hash Cond: (e.dept_id = d.dept_id)
        ->  Hash Anti Join  (cost=1.18..2.39)
              Hash Cond: (e.emp_id = ep.emp_id)
              ->  Seq Scan on employee e
                  (cost=0.00..1.11)
              ->  Hash  (cost=1.08..1.08)
                    ->  Seq Scan on emp_projects ep
                        (cost=0.00..1.08)
        ->  Hash  (cost=1.04..1.04)
              ->  Seq Scan on department d
                  (cost=0.00..1.04)
```

Query plans — EXPLAIN

Ten istý dotaz pri 100 000 riadkoch v každej tabuľke:

```
HashAggregate  (cost=9653.28..10117.69)
  Group Key: d.dept_id
  ->  Hash Join  (cost=6178.00..9421.07)
        Hash Cond: (e.dept_id = d.dept_id)
        ->  Hash Anti Join
            (cost=2693.00..5814.16)
              Hash Cond: (e.emp_id = ep.emp_id)
              ->  Seq Scan on employee e
                  (cost=0.00..2334.00)
              ->  Hash  (cost=1443.00..1443.00)
                    ->  Seq Scan on emp_projects ep
                        (cost=0.00..1443.00)
        ->  Hash  (cost=2235.00..2235.00)
              ->  Seq Scan on department d
                  (cost=0.00..2235.00)
```

Query plans — EXPLAIN

Ten istý dotaz pri 1 000 000 riadkoch v každej tabuľke:

```
Finalize GroupAggregate  (cost=84641.74..142659.79)
  Group Key: d.dept_id
  ->  Gather Merge  (cost=84641.74..135673.37)
        Workers Planned: 2
        ->  Partial GroupAggregate  (cost=83641.72..87237.67)
              Group Key: d.dept_id
              ->  Sort  (cost=83641.72..84155.42)
                    Sort Key: d.dept_id
                    ->  Parallel Hash Join  (cost=43767.37..62698.72)
                          Hash Cond: (d.dept_id = e.dept_id)
                          ->  Parallel Seq Scan on department d
                              (cost=0.00..16512.67)
                          ->  Parallel Hash  (cost=41198.83..41198.83)
                                ->  Parallel Hash Anti Join
                                    (cost=15428.00..41198.83)
                                      Hash Cond: (e.emp_id = ep.emp_id)
                                      ->  Parallel Seq Scan on employee e
                                          (cost=0.00..17500.67)
                                      ->  Parallel Hash  (cost=8591.67..8591.67)
                                            ->  Parallel Seq Scan on emp_projects ep
                                                (cost=0.00..8591.67)
```

Paralelná agregácia: Partial a Finalize

- Procesy si rozdelia spracovanie vstupu. Každý vypočíta čiastkový výsledok pre skupiny, ktoré našiel. To isté oddelenie sa môže vyskytnúť vo výsledkoch viacerých procesov.
- `Gather` alebo `Gather Merge` zozbiera čiastkové výsledky.
- `Finalize GroupAggregate` ich skombinuje: pri `COUNT` sčíta čiastkové počty. Ak jeden proces pre oddelenie 10 napočítal 30 zamestnancov a druhý 45, výsledok je 75.

`Partial` označuje čiastkový výsledok paralelného výpočtu; `HashAggregate` a `GroupAggregate` označujú spôsob agregácie.

Query plans — EXPLAIN

Počet zamestnancov bez projektu na oddeleniach s `dept_id <= 100`.

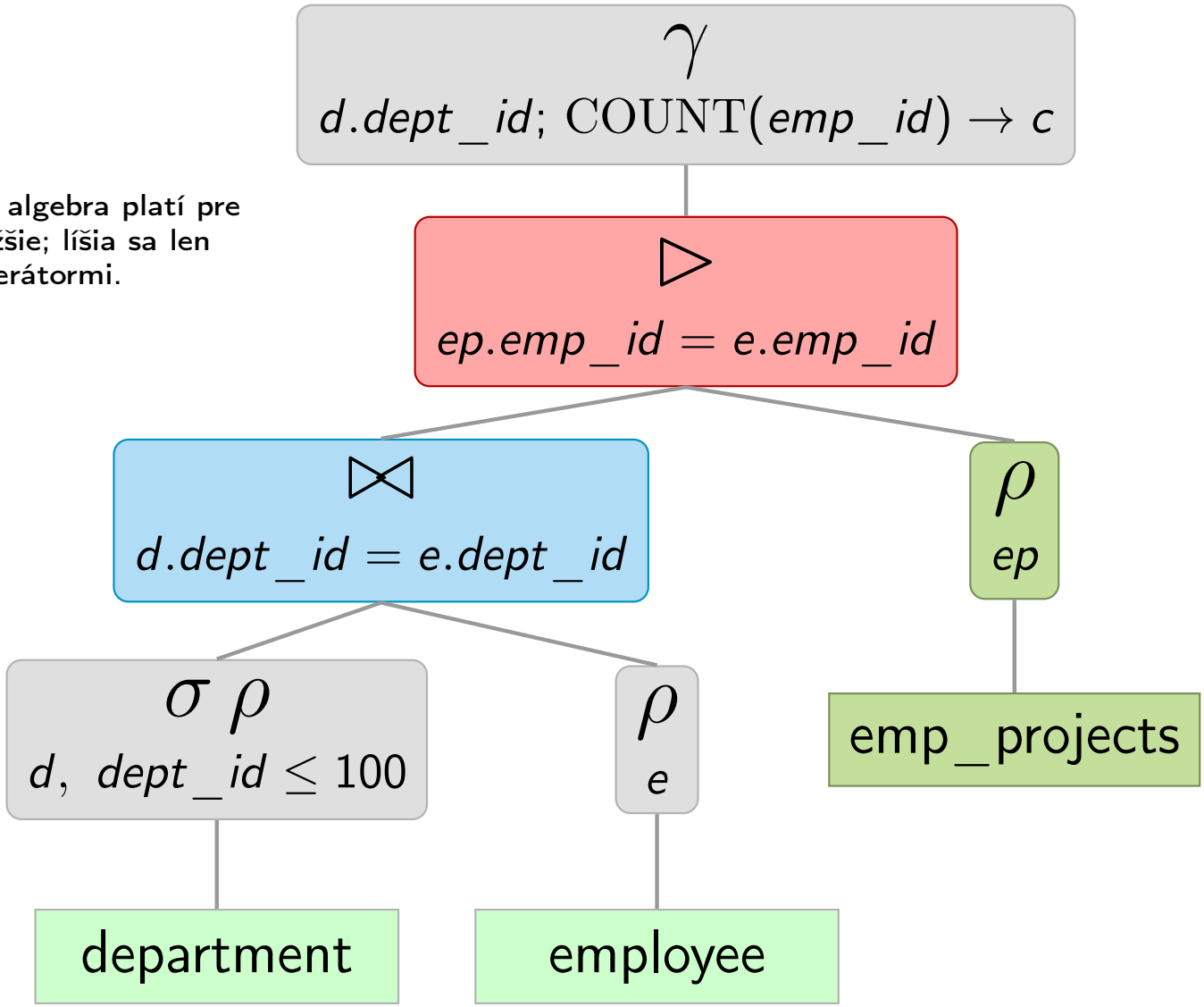
Hash index na `emp_projects(emp_id)` pri dotaze s výberom oddelení (filter `dept_id` ≤ 100, 1 000 000 záznamov v každej tabuľke):

```
SELECT d.dept_id, COUNT(e.emp_id) AS c
FROM department d
JOIN employee e ON e.dept_id = d.dept_id
WHERE d.dept_id <= 100
  AND NOT EXISTS (
      SELECT 1
      FROM emp_projects ep
      WHERE ep.emp_id = e.emp_id
  )
GROUP BY d.dept_id;
```

```
CREATE INDEX ON emp_projects USING HASH (emp_id)
```

Query plans — EXPLAIN

Táto relačná algebra platí pre oba plány nižšie; líšia sa len fyzickými operátormi.



Query plans — EXPLAIN

Plán bez indexu:

```
Finalize GroupAggregate  (cost=29756.72..29762.08)
  Group Key: d.dept_id
  ->  Gather Merge  (cost=29756.72..29761.44)
    Workers Planned: 2
    ->  Partial GroupAggregate  (cost=28756.70..28757.03)
      Group Key: d.dept_id
      ->  Sort  (cost=28756.70..28756.74)
        Sort Key: d.dept_id
        ->  Parallel Hash Right Anti Join  (cost=18601.98..28756.29)
          Hash Cond: (ep.emp_id = e.emp_id)
          ->  Parallel Seq Scan on emp_projects ep  (cost=0.00..8591.67)
          ->  Parallel Hash  (cost=18601.51..18601.51)
            ->  Hash Join  (cost=7.09..18601.51)
              Hash Cond: (e.dept_id = d.dept_id)
              ->  Parallel Seq Scan on employee e  (cost=0.00..17500.67)
              ->  Hash  (cost=5.98..5.98)
                ->  Index Only Scan using department_pkey on department d  (cost=0.42..5.98)
                  Index Cond: (dept_id <= 100)
```

Query plans — EXPLAIN

Plán s hash indexom:

```
Finalize GroupAggregate  (cost=19603.67..19608.76)
  Group Key: d.dept_id
  ->  Gather Merge  (cost=19603.67..19608.14)
    Workers Planned: 2
    ->  Partial GroupAggregate  (cost=18603.65..18603.97)
      Group Key: d.dept_id
      ->  Sort  (cost=18603.65..18603.70)
        Sort Key: d.dept_id
        ->  Nested Loop Anti Join  (cost=7.09..18603.28)
          ->  Hash Join  (cost=7.09..18601.51)
            Hash Cond: (e.dept_id = d.dept_id)
            ->  Parallel Seq Scan on employee e  (cost=0.00..17500.67)
            ->  Hash  (cost=5.98..5.98)
              ->  Index Only Scan using department_pkey on department d  (cost=0.42..5.98)
                Index Cond: (dept_id <= 100)
          ->  Index Scan using emp_projects_emp_id_idx on emp_projects ep  (cost=0.00..0.07)
            Index Cond: (emp_id = e.emp_id)
```

Query plans — EXPLAIN

Čo sa v pláne zmenilo:

- Červená: zmenený operátor (hash anti join na nested loop anti join) a celkový odhad času (cost).
- Modrá: podstrom `employee` ⋈ `department` (zamestnanci oddelení `dept_id <= 100`). Bez indexu sa tento výsledok hašuje (`Parallel Hash`) a oproti nemu sa prechádza `emp_projects`. S indexom sa zamestnanci prechádzajú jeden po druhom ako vonkajší vstup (`Nested Loop`).
- Zelená: prístup k `emp_projects`. Bez indexu sa celá tabuľka prečíta sekvenčne (`Parallel Seq Scan`); s indexom sa čítajú len záznamy konkrétneho zamestnanca cez `Index Scan`.
- Hash index nepomáha `Hash Joinu` (ten si hašovaciu tabuľku vytvorí sám). Skôr urýchľuje `Nested Loop` (tu pri vyhľadávaní v `emp_projects`).

Query plans — EXPLAIN

- (ilustratívne čísla, nie namerané hodnoty) Plánovanie trvalo prvýkrát 22 ms, pri opakovanom výpočte toho istého dotazu už len 0.5 ms (detaily pre PostgreSQL na ďalšom slide).
- Plán závisí od vstupov optimalizácie (štatistiky o dátach, hodnoty parametrov, nastavenia), preto sa môže zmeniť, aj keď sa samotný dotaz nezmenil (napr. po ANALYZE). Pri veľkom počte joinovaných relácií sa namiesto úplného prehľadávania často používa heuristika alebo genetický algoritmus (napr. PostgreSQL od 12 relácií).
- Väčšina práce pri výpočte dotazu spočíva v usporadúvaní, spájaní a hashovaní (sort / merge / hash), znova a znova.

Prepared statements

Dotaz možno pripraviť vopred a potom ho opakovane vykonávať s rôznymi parametrami.

```
PREPARE emp_by_dept (int) AS
    SELECT name FROM employee WHERE dept_id = $1;

EXECUTE emp_by_dept(7);
EXECUTE emp_by_dept(3);

DEALLOCATE emp_by_dept;
```

PostgreSQL: ad hoc dotaz sa optimalizuje pri každom spustení. Prepared statement môže po niekoľkých vykonaniach použiť výsledok optimalizácie uložený v cache, takže sa optimalizácia neopakuje. Vytvorený plán nemusí fungovať dobre pre každú hodnotu parametrov, napr. ak hodnota parametra vyberá z nejakej tabuľky veľmi rôzne počty riadkov.

Query plans — MAX a ORDER BY s LIMIT

Zjednodušené plány pre väčšiu neprázdnu tabuľku bez indexu na `salary`; platy nie sú `NULL`.

```
EXPLAIN ANALYZE
SELECT MAX(salary)
FROM employee;

Aggregate
  -> Seq Scan on employee
```

```
EXPLAIN ANALYZE
SELECT salary FROM employee
ORDER BY salary DESC LIMIT 1;

Limit
  -> Sort
       Sort Key: salary DESC
       Sort Method: top-N heapsort
       -> Seq Scan on employee
```

- Pre LIMIT 1 si top-N heapsort ponecháva najlepší doterajší riadok; nemusí uchovať ani utriediť celý vstup.
- Oba varianty prejdú všetky riadky, majú lineárnu časovú zložitosť a v tomto prípade bývajú približne rovnako rýchle.

Query plans — MAX a ORDER BY s indexom

B-tree index `i_salary` na stĺpci `salary`:

`MAX(salary)`

```
Result
  InitPlan 1 (returns $0)
    -> Limit
      -> Index Only Scan Backward
           using i_salary
```

`ORDER BY ... LIMIT 1`

```
Limit
  -> Index Only Scan Backward
       using i_salary
```

- Oba varianty vyhľadajú najväčšiu hodnotu cez index a skončia po jednom výslednom riadku; neprechádzajú celú tabuľku.
- Namiesto lineárneho prechodu stačí logaritmické vyhľadanie v B-tree: pri veľkej tabuľke môže byť zrýchlenie obrovské.

Literatúra

B+ stromy a indexy

- https://cs186berkeley.net/notes/note4/
- https://www.db-book.com/slides-dir/PDF-dir/ch14.pdf

Optimalizácia dotazov

- https://cs186berkeley.net/notes/note10/
- https://www.db-book.com/slides-dir/PDF-dir/ch16.pdf
- https://pganalyze.com/docs/explain/scan-nodes
- https://www.postgresql.org/docs/current/performance-tips.html

Na zamyslenie

- Prečo sa v databázových indexoch nepoužívajú binárne stromy?
- Aké výhody a nevýhody má pridanie indexu do databázy?
- Uveďte príklad dotazu, pri ktorom by použitie indexu spomalilo výpočet (hoci index je navrhnutý dobre a pri iných dotazoch slúži, ako má).
- Ako funguje *bitmap index scan*? (PostgreSQL: link)
- Pred samotným výpočtom dotazu sa vykoná veľa krokov (napr. parsovanie dotazu a preklad do relačnej algebry). Ktorý z týchto krokov trvá zvyčajne najdlhšie?
- Ako možno na úrovni relačnej algebry (bez skúmania dát v tabuľkách) optimalizovať operátory σ a π?